

ALU Brand Protection dApp

GitHub Repository: <https://github.com/aimable0/alu-logo-protection/>

Demo Video: <https://youtu.be/EJrRTBUGJt8>

Abstract

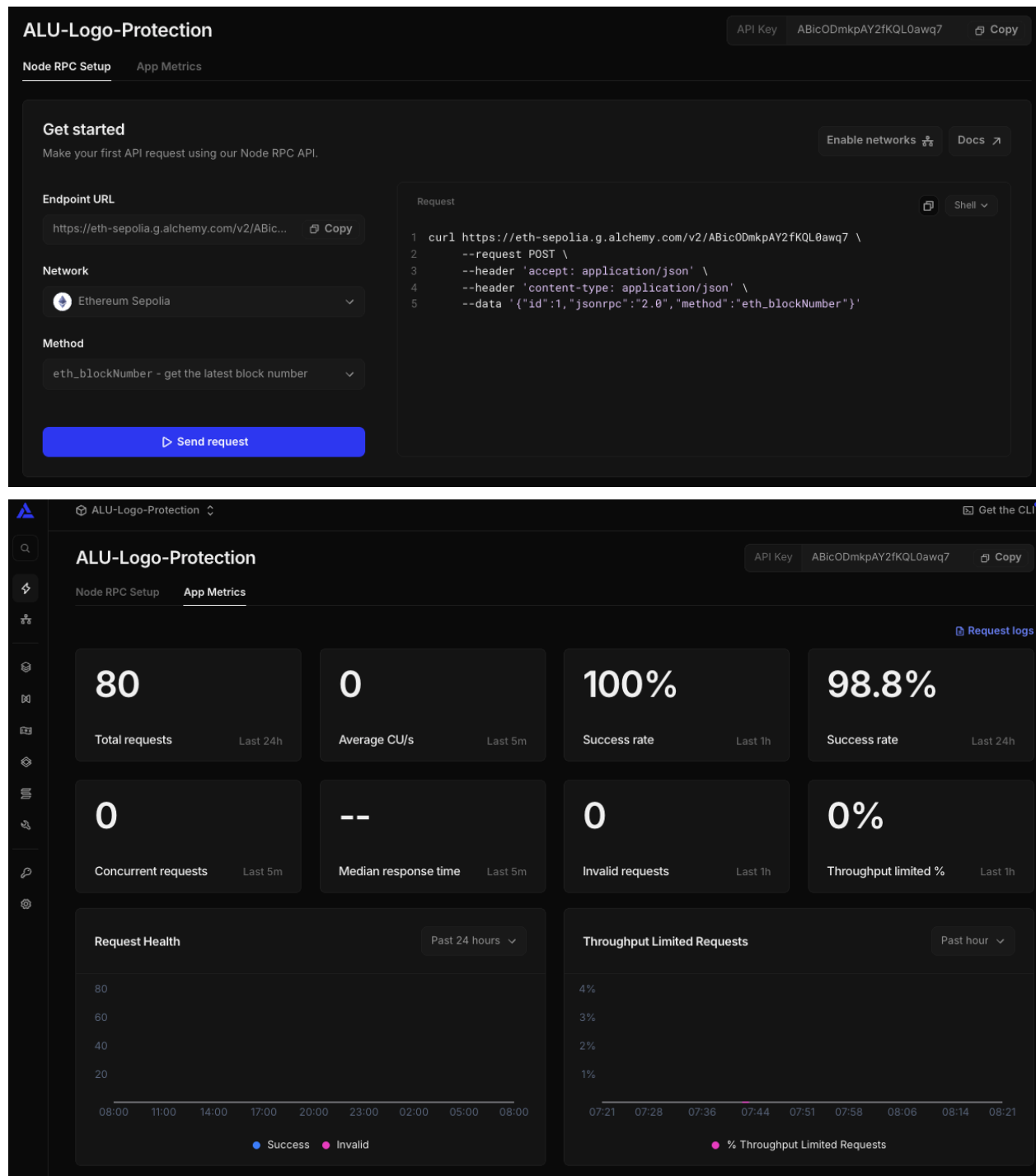
This decentralized application (dApp) protects the intellectual property of the African Leadership University (ALU) logo by establishing an unforgeable cryptographic record on the blockchain. It solves the problem of digital counterfeiting by using an ERC-721 registry to verify authenticity, and an ERC-20 token contract to distribute fractionalized ownership. Together, these contracts create a secure, transparent ecosystem for managing institutional brand assets.

Introduction

While smart contracts are highly secure, they are inherently inaccessible to non-technical users who cannot read code or execute command-line scripts. This dApp fills that gap by providing an intuitive graphical interface. It bridges complex blockchain logic with everyday administrative needs, allowing staff to easily register assets and the general public to verify files without requiring any programming knowledge.

Architecture Overview

The architecture is divided into three layers: the frontend, the provider, and the smart contracts. The frontend is built using React to manage the user interface. It utilizes Ethers.js as the 'bridge' to communicate with both the local Hardhat network during automated testing and the live **Ethereum Sepolia Testnet** for deployment. For actions that change the blockchain state, the frontend connects to the user's MetaMask wallet to request transaction signatures. For public, read-only actions (such as verifying a logo hash), the frontend utilizes an **Alchemy JsonRpcProvider** to query the Sepolia testnet directly without requiring a wallet connection.



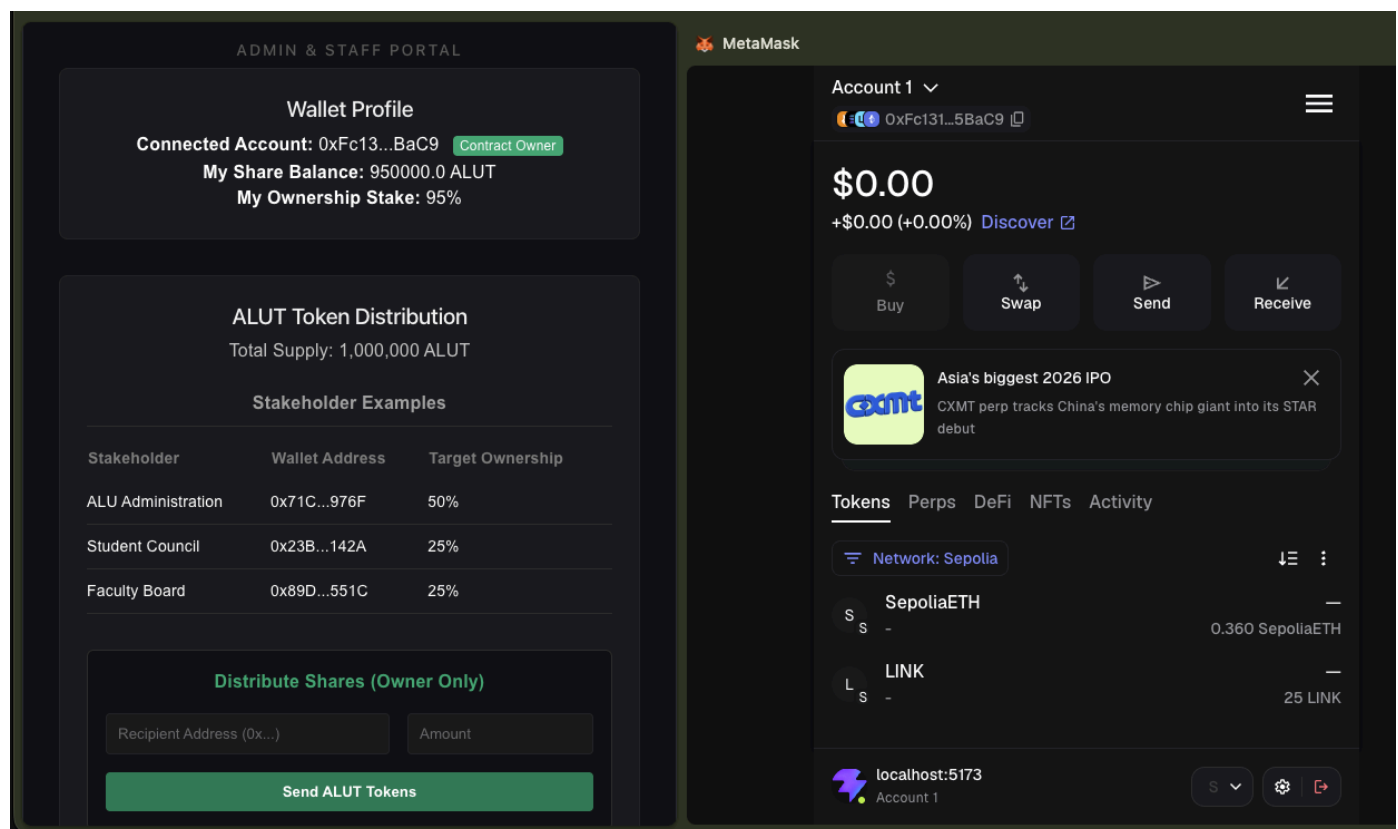
Wallet and Web3 Integration

Wallet connection works by accessing the Ethereum provider (MetaMask) injected into the browser. Ethers.js wraps this provider, allowing the application to send specific commands to the network.

To facilitate this, the frontend relies on an ABI (Application Binary Interface). The ABI is essentially a translation manual; it tells the frontend exactly what functions exist in the smart contract and what data types they expect, allowing the JavaScript code to format its requests correctly.

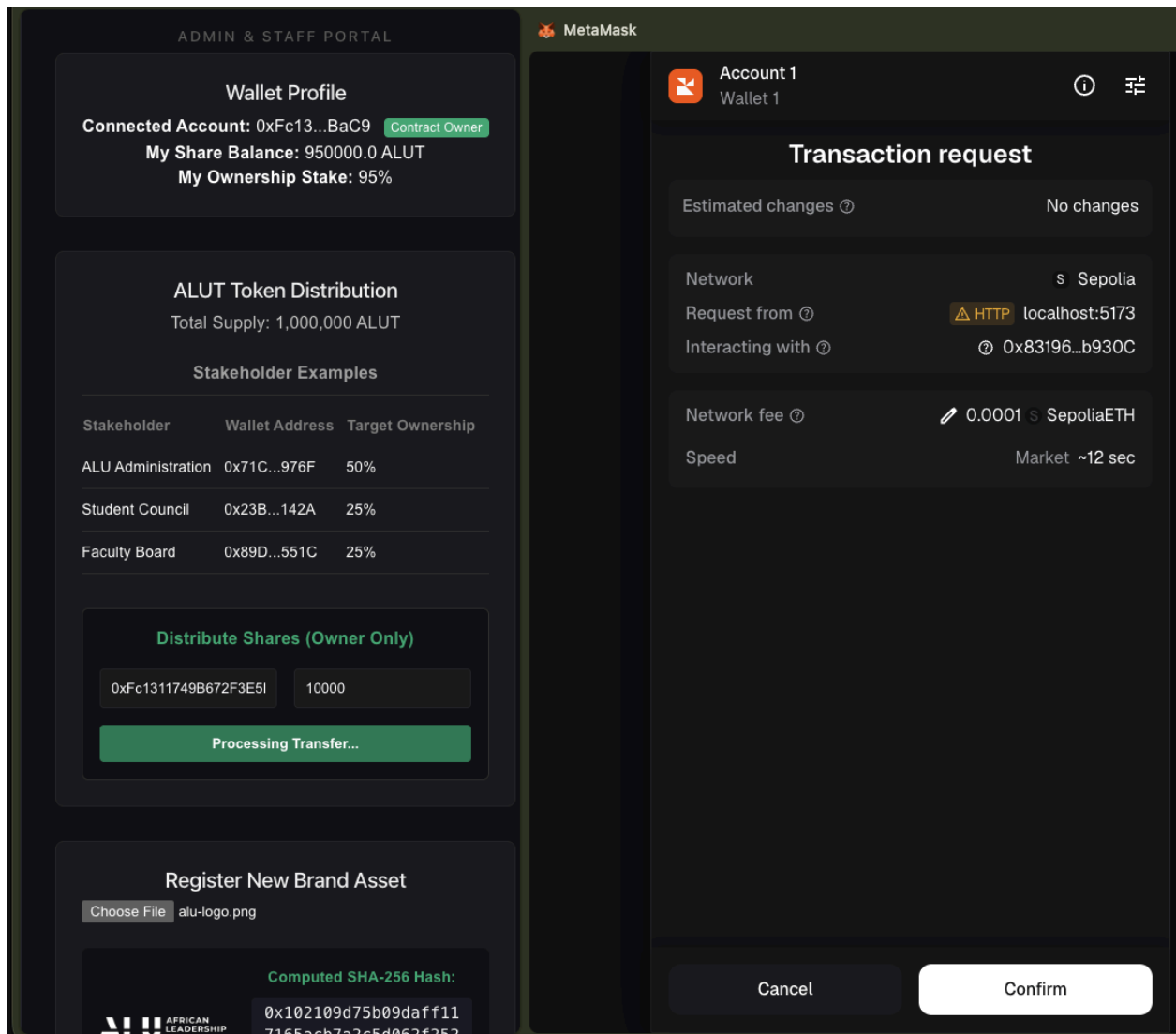
The dApp executes two distinct types of calls. A read-only call simply reads existing data from the network. Because it does not alter the blockchain, it requires no network validation, no wallet signature, and costs zero gas. Conversely, a transaction alters the blockchain's state (such as transferring tokens). This requires computational work from the network, meaning it costs gas fees and requires a cryptographic signature.

As seen in here below,



the user must explicitly sign these transactions using their own wallet rather than the application signing on their behalf. This fundamental Web3 security principle ensures absolute user custody; the dApp cannot maliciously spend the user's funds or execute unauthorized actions.

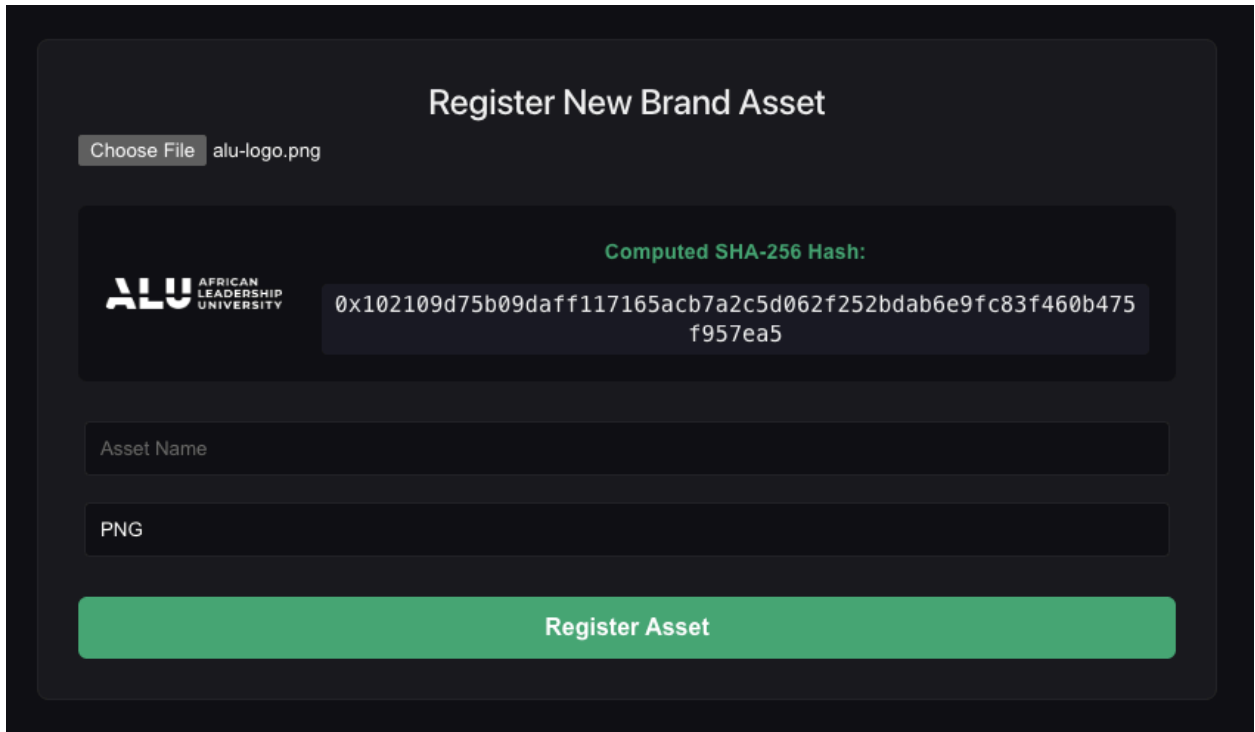
Sample below:



Feature Walkthrough (Asset Registration)

The asset registration portal allows administrators to upload a brand asset and compute its cryptographic identity.

As shown in sample image below:



Register New Brand Asset

Choose File alu-logo.png

ALU AFRICAN LEADERSHIP UNIVERSITY

Computed SHA-256 Hash:

0x102109d75b09da7f117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5

Asset Name

PNG

Register Asset

the application generates a SHA-256 hash. It is critical that this hash is generated in the browser rather than sending the file to a central server. Doing this locally preserves the privacy of unreleased assets and prevents bad actors from intercepting the file in transit.

When the user clicks to register the asset, MetaMask asks them to approve the transaction. By approving, the user is cryptographically agreeing to broadcast the payload to the network and pay the required gas fee. If a user attempts to register a file that is already in the system.

Sample Image below:

Register New Brand Asset

Choose File

alu-logo.png



Computed SHA-256 Hash:
0x102109d75b09daff117165acb7a2c5d062f252bdab6e9fc83f460b475f957ea5

ALU-Logo

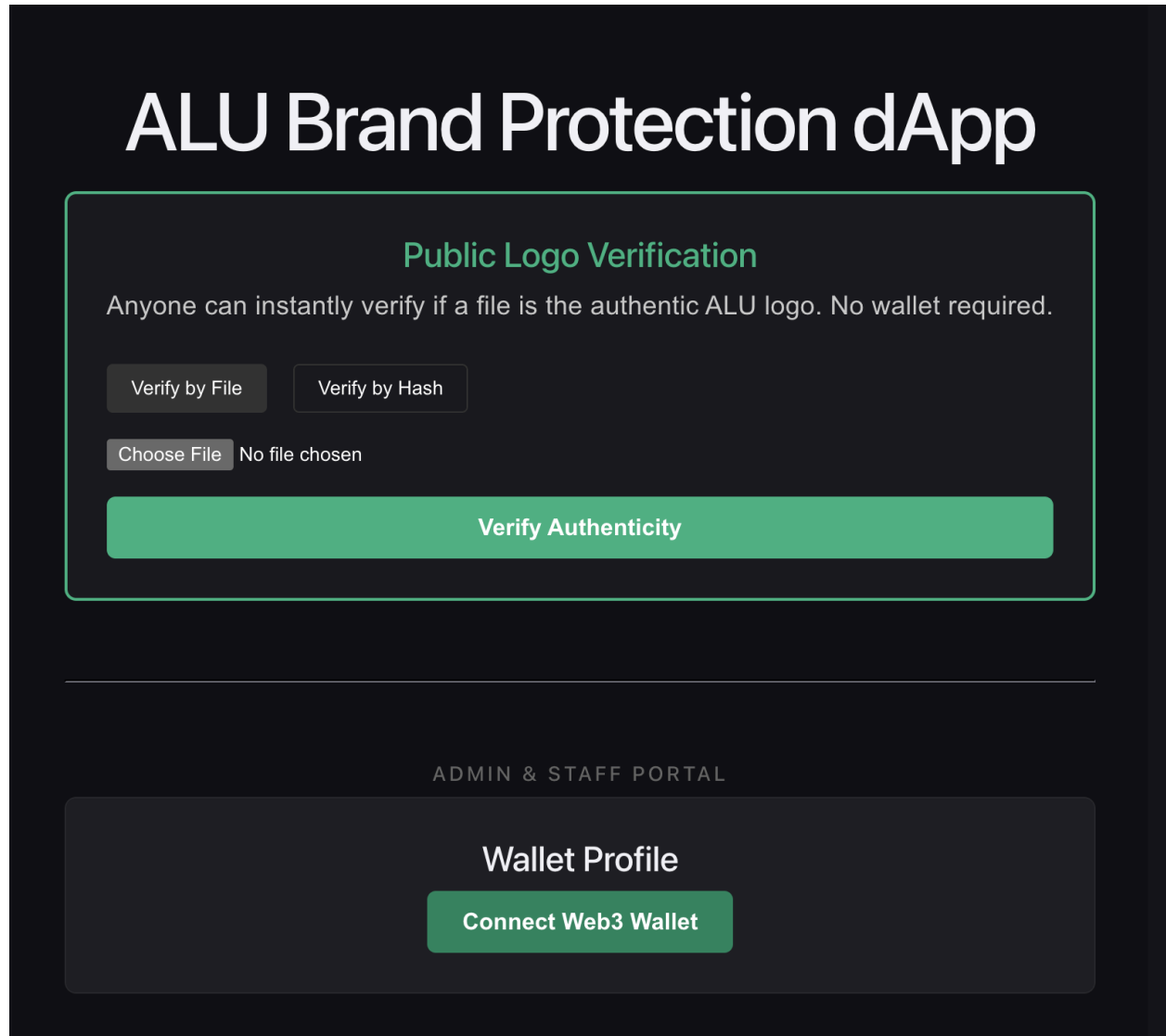
PNG

Register Asset

Error: This content hash has already been registered

the smart contract's internal `require` statement immediately triggers a revert, terminating the transaction at the contract level to prevent duplicate ownership claims.

Verification Page



The image shows a dark-themed web interface for the 'ALU Brand Protection dApp'. At the top, the title 'ALU Brand Protection dApp' is displayed in large white font. Below it, a section titled 'Public Logo Verification' in green text explains that anyone can verify a file's authenticity without a wallet. Two buttons, 'Verify by File' and 'Verify by Hash', are shown. The 'Verify by File' button is active, leading to a file selection interface with a 'Choose File' button and the text 'No file chosen'. A large green 'Verify Authenticity' button is positioned below the file selection area. A horizontal line separates this section from the 'ADMIN & STAFF PORTAL' section below. This section features a 'Wallet Profile' heading and a green 'Connect Web3 Wallet' button.

ALU Brand Protection dApp

Public Logo Verification

Anyone can instantly verify if a file is the authentic ALU logo. No wallet required.

Verify by File Verify by Hash

Choose File No file chosen

Verify Authenticity

ADMIN & STAFF PORTAL

Wallet Profile

Connect Web3 Wallet

The public verification page allows anyone to check a file's authenticity. This page does not require a wallet connection because the `verifyLogoIntegrity()` function is a `view` function (a read-only call). It merely looks up existing data on the blockchain, which costs no gas and requires no user identity.

A real-world scenario for this feature involves a third-party print shop contracted to create ALU merchandise. If they receive a logo file via an email thread, they can upload it to this portal. This solves the problem of version control and tampering, ensuring they are using the exact, university-approved asset. The print shop can inherently trust the result shown by the dApp

alu-logo.png

ALU Brand Protection dApp

Public Logo Verification

Anyone can instantly verify if a file is the authentic ALU logo. No wallet required.

Verify by File

Verify by Hash

Choose File alu-logo.png

Verify Authenticity

✓ Logo Verified: Authentic

This file mathematically matches the official ALU logo registered on the blockchain.

- **Asset Name:** ALU Logo
- **Registered On:** 7/18/2026, 10:39:16 AM

ALU Brand Protection dApp

Public Logo Verification

Anyone can instantly verify if a file is the authentic ALU logo. No wallet required.

Verify by File

Verify by Hash

Choose File Screenshot 2026-07-18 at 11.11.14AM

Verify Authenticity

 **Warning: Invalid Logo**

This file has been modified or is a fake. It does not match the authentic ALU records.

because blockchains are immutable; the cryptographic record cannot be secretly altered, even by ALU administrators.

Token Dashboard

The dashboard displays the connected user's token balance and provides a secure interface for distributing equity

Wallet Profile

Connected Account: 0xFc13...BaC9 Contract Owner

My Share Balance: 950000.0 ALUT

My Ownership Stake: 95%

ALUT Token Distribution

Total Supply: 1,000,000 ALUT

Stakeholder Examples

Stakeholder	Wallet Address	Target Ownership
ALU Administration	0x71C...976F	50%
Student Council	0x23B...142A	25%
Faculty Board	0x89D...551C	25%

Distribute Shares (Owner Only)

Send ALUT Tokens

Register New Brand Asset

Choose File No file chosen

The `distributeShares()` function is protected by an `onlyOwner` modifier. If a non-owner wallet attempts to call this function, the transaction reverts at the contract level before any state changes occur, protecting the token supply from unauthorized transfers.

A practical example of this distribution would be ALU sending 25% of the ALUT tokens to the Student Council. In practice, holding these tokens could grant the council proportional voting rights on brand-related decisions, such as approving new merchandise designs. To implement this proportional voting in the future, the smart contract would need to be updated to inherit ERC-20 governance extensions (like OpenZeppelin's `ERC20Votes`), which add snapshot checkpointing to track historical voting power based on token balances.

Test Results

The automated Hardhat test suite contains tests covering the ERC-721 registry, the ERC-20 token, and the frontend integrations.

```
Aimables-MacBook-Pro:alu-logo-protection mac$ npx hardhat test
No contracts to compile

Running Solidity tests

Running Mocha tests

ALU Final Project Contracts
Part B: ALUAssetRegistry (ERC-721)
  ✓ Registers the ALU logo successfully and returns a token ID
  ✓ Attempting to register the same hash a second time is rejected with an error
  ✓ verifyLogoIntegrity() returns true when the correct hash is supplied
  ✓ verifyLogoIntegrity() returns false when an incorrect hash is supplied
  ✓ getAsset() returns the correct asset name and file type for a registered token
Part C: ALULogoToken (ERC-20)
  ✓ Mints the full supply of 1,000,000 ALUT tokens to the logo owner upon deployment
  ✓ distributeShares() correctly transfers the specified number of tokens to a recipient address
  ✓ ownershipPercentage() returns the correct percentage for a wallet that holds a known number of tokens
Frontend Integration & Distribution Tests
  ✓ 1. Should correctly read the total ALUT token supply as 1,000,000
  ✓ 2. Should format the SHA-256 hash correctly in bytes32 format
  ✓ 3. Should return a verification success result for the correct ALU logo hash
  ✓ 4. Should return a verification failure result for an incorrect hash
  ✓ 5. Should correctly update the recipient's balance after distributeShares()

13 passing (442ms)

13 passing (13 mocha)
```

Note: "Before deploying to Sepolia, the full contract suite was validated locally using Hardhat's simulated network to ensure 100% test coverage for state changes, revert conditions, and access controls."

As seen in the terminal output above, the system passes all 13 integration and unit tests, validating the core logic, duplicate rejection mapping, and token transfer math.

Conclusion

Building this full-stack Web3 application provided invaluable hands-on experience in bridging decentralized smart contract backends with modern React interfaces using Ethers.js. Deploying and verifying the smart contracts on the live **Ethereum Sepolia Testnet** successfully transitioned the project from a local prototype to a production-ready Web3 application.

Live Sepolia Contract Deployments:

- **ALUAssetRegistry (ERC-721):**
[0x258Ca10734fa6A203B44E9202BC0b02c3f2f76Fe](#)
- **ALULogoToken (ERC-20):**
[0x83196fF6820E94e66f706a49B5E36808575b930C](#)

As I prepare to finish my university classes later this year and pursue a career as a software engineer, mastering end-to-end dApp development—from Solidity development and automated Hardhat testing to live RPC deployment—is a critical milestone. For future iterations, the dApp could be further enhanced by integrating an IPFS (InterPlanetary File System) storage solution so the smart contracts point to decentralized, hosted image files rather than just tracking content hashes, alongside inheriting OpenZeppelin ERC20Votes extensions for on-chain brand governance.